Apuntes de Programación en Python · Modelo de examen

Apuntes resueltos del modelo de examen: **enunciado** → **mi solución** → **explicación**. Cubre clases y objetos, comprehensions, programación funcional, recursión, sobrecarga de operadores, herencia (simple y múltiple), refactorización y programación modular.

Leyenda

- ✅ Mi código era correcto.
- ⚠️ Mi código tenía algún fallo: aquí aparece la **versión corregida** y qué falló.
- 🔁 **Refuerzo**: idea que conviene repasar antes del examen.

Índice

1. Lectura de código
2. Comprehensions
3. Programación funcional
4. Recursión
5. Clases y objetos
6. Sobrecarga de operadores y conversión de tipos
7. Herencia, polimorfismo y herencia múltiple
8. Refactorización: de código espagueti a clases
9. Ejercicio integrador: restaurante
10. Programación modular
11. Resumen de errores a evitar
12. Chuleta final

Sección 1. Lectura de código: ¿qué se muestra por pantalla?

No hay que programar: se lee el código, se predice la salida y después se ejecuta para comprobarla.

Ejercicio 1.1 · El contador de tiendas

```
class Tienda:
    total_tiendas = 0

    def __init__(self, nombre):
        self.nombre = nombre
        Tienda.total_tiendas += 1

t1 = Tienda("Centro")
t2 = Tienda("Norte")
t1.total_tiendas = 10
```

```
print(Tienda.total_tiendas, t1.total_tiendas, t2.total_tiendas)
```

Pregunta extra: ¿por qué `t1` y `t2` no muestran el mismo número?

Salida

```
2 10 2
```

Explicación

- `Tienda.total_tiendas` vale `2`: el constructor lo incrementó una vez por cada tienda creada.
- `t1.total_tiendas = 10` **no modifica el atributo de clase**. Crea un atributo de instancia nuevo en `t1` que "tapa" al de la clase.
- `t2` no tiene atributo propio, así que Python lo busca en la clase y encuentra `2`.
- Para modificar el contador compartido hay que usar `Tienda.total_tiendas = ...`.

> 🔁 **Refuerzo:** al leer `objeto.atributo`, Python busca primero en la instancia y después en la clase. Al **escribir** `objeto.atributo = valor`, siempre crea o modifica el de la instancia.

Ejercicio 1.2 · Dos listas que parecen tres

```
pedidos_a = [101, 102, 103]
pedidos_b = pedidos_a
pedidos_c = pedidos_a[:]

pedidos_b.append(104)
pedidos_c.append(999)

print(pedidos_a)
print(pedidos_b)
print(pedidos_c)
print(pedidos_a is pedidos_b, pedidos_a is pedidos_c, pedidos_a == pedidos_b)
```

Salida

```
[101, 102, 103, 104]
[101, 102, 103, 104]
[101, 102, 103, 999]
True False True
```

Explicación

- `pedidos_b = pedidos_a` **no copia**: son dos nombres para la misma lista, así que el `append(104)` se ve en ambas.
- `pedidos_a[:]` crea una **copia independiente**, así que el `999` solo afecta a `pedidos_c`.
- `is` compara **identidad** (¿mismo objeto?); `==` compara **contenido**.

Ejercicio 1.3 · Filtrar o clasificar

```
valoraciones = [4, 7, 5, 9, 3]

resultado_1 = [v for v in valoraciones if v >= 5]
resultado_2 = ["POSITIVA" if v >= 5 else "NEGATIVA" for v in valoraciones]

print(resultado_1)
print(resultado_2)
```

Salida

```
[7, 5, 9]
['NEGATIVA', 'POSITIVA', 'POSITIVA', 'POSITIVA', 'NEGATIVA']
```

Pregunta extra: ¿por qué el `if` está en una posición distinta? Son dos usos distintos:

- Al **final** es un **filtro**: decide qué elementos entran en la lista. El resultado puede ser más corto.
- Al **principio** es una **expresión condicional**: decide qué valor se genera para cada elemento. Necesita `else` porque siempre tiene que producir algo, y la lista resultante tiene la misma longitud que la original.

Ejercicio 1.4 · Cuenta atrás

```
def cuenta_atras(n):
    if n == 0:
        return "¡Despegue!"
    return str(n) + " " + cuenta_atras(n - 1)

print(cuenta_atras(3))
```

Salida

```
3 2 1 ¡Despegue!
```

Desarrollo de la recursión

```
cuenta_atras(3) = "3 " + cuenta_atras(2)
                = "3 " + "2 " + cuenta_atras(1)
                = "3 " + "2 " + "1 " + cuenta_atras(0)
                = "3 " + "2 " + "1 " + "¡Despegue!"
```

Pregunta extra: ¿qué pasa si se borran las dos líneas del `if`? Desaparece el **caso base**. La función se llamaría con `n = 3, 2, 1, 0, -1, -2...` sin parar nunca, hasta agotar el límite de recursión de Python y lanzar un `RecursionError` (*maximum recursion depth exceeded*).

Ejercicio 1.5 · Recepción de un hotel

```python
class Huesped:
    def bienvenida(self):
        return "Bienvenido"

class HuespedVIP(Huesped):
    def bienvenida(self):
        return super().bienvenida() + " - Tiene champán en la habitación"

class HuespedGrupo(Huesped):
    def bienvenida(self):
        return "Bienvenidos, el guía les espera en el vestíbulo"

for huesped in [Huesped(), HuespedVIP(), HuespedGrupo()]:
    print(huesped.bienvenida())
```

Salida

```
Bienvenido
Bienvenido - Tiene champán en la habitación
Bienvenidos, el guía les espera en el vestíbulo
```

Explicación

- `Huesped` usa su propio método.
- `HuespedVIP` llama con `super()` al método del padre y le **añade** texto (*extiende* el comportamiento).
- `HuespedGrupo` **sobrescribe** el método por completo, sin usar el del padre.
- Esto es **polimorfismo**: el mismo mensaje (`bienvenida()`) produce respuestas distintas según la clase del objeto.

Ejercicio 1.6 · Atributos con guiones bajos

```
class Tarjeta:
    def __init__(self):
        self.titular = "Ana"
        self._saldo = 500
        self.__pin = 1234

t = Tarjeta()
print(t.titular)        # Línea A
print(t._saldo)         # Línea B
print(t._Tarjeta__pin)  # Línea C
print(t.__pin)          # Línea D
```

Respuesta

Línea	Resultado
A	Funciona: imprime `Ana`
B	Funciona: imprime `500` (el guion bajo simple es solo una **convención**)
C	Funciona: imprime `1234` (se accede con el nombre real tras el *name mangling*)
D	**Error** `AttributeError`: `'Tarjeta' object has no attribute '__pin'`

Python renombra internamente `__pin` a `_Tarjeta__pin` (*name mangling*), así que `__pin` no existe desde fuera.

> 🔁 **Refuerzo:** `_x` = "uso interno" (solo convención). `__x` = Python lo renombra a `_Clase__x`. Ninguno de los dos es privacidad real.

Ejercicio 1.7 · Funciones que fabrican funciones

```
def crear_etiqueta(prefijo):
    def etiquetar(codigo):
        return prefijo + "-" + str(codigo)
    return etiquetar

factura = crear_etiqueta("FAC")
albaran = crear_etiqueta("ALB")

print(factura(15), albaran(7), factura(albaran(1)))
```

Salida

```
FAC-15 ALB-7 FAC-ALB-1
```

Explicación

- `crear_etiqueta("FAC")` devuelve la función `etiquetar`, que **recuerda** `prefijo = "FAC"`. Esto es una **clausura** (*closure*).
- `factura(albaran(1))`: primero `albaran(1)` da `"ALB-1"`, y después `factura("ALB-1")` da `"FAC-" + "ALB-1"` = `"FAC-ALB-1"`.

Ejercicio 1.8 · La cafetería y la herencia múltiple

```
class Bebida:
    def ingredientes(self):
        return ["café"]

class ConLeche(Bebida):
    def ingredientes(self):
        return super().ingredientes() + ["leche"]

class ConHielo(Bebida):
    def ingredientes(self):
        return super().ingredientes() + ["hielo"]

class Frappe(ConLeche, ConHielo):
    def ingredientes(self):
        return super().ingredientes() + ["nata"]

class CafeFrio(ConHielo, ConLeche):
    pass

print(Frappe().ingredientes())
print(CafeFrio().ingredientes())
print([clase.__name__ for clase in Frappe.__mro__])
```

Salida

```
['café', 'hielo', 'leche', 'nata']
['café', 'leche', 'hielo']
['Frappe', 'ConLeche', 'ConHielo', 'Bebida', 'object']
```

1. Dentro de `ConLeche`, `super()` no llama a `Bebida`. ¿A qué clase llama en un `Frappe` y por qué? Llama a `ConHielo`. `super()` no significa "mi clase padre", sino **"la siguiente clase en el MRO del objeto"**. El MRO de `Frappe` es `Frappe → ConLeche → ConHielo → Bebida → object`.

2. ¿Por qué `"café"` aparece una sola vez? Porque `Bebida` aparece **una sola vez en el MRO**, y su método se ejecuta una sola vez. Es la solución al **problema del diamante**.

3. ¿Por qué cambia el orden en `CafeFrio`? Porque el MRO es distinto: `CafeFrio → ConHielo → ConLeche → Bebida → object`. Las llamadas bajan por el MRO, pero cada método **añade su ingrediente al volver**. Por

eso el orden de la lista es el **MRO leído al revés**:

Objeto	MRO	Lista (MRO al revés)
`Frappe`	Frappe → ConLeche → ConHielo → Bebida	café, hielo, leche, nata
`CafeFrio`	CafeFrio → ConHielo → ConLeche → Bebida	café, leche, hielo

> 🔁 **Refuerzo:** `super()` = "el siguiente en el MRO", no "el padre". Para ver el MRO: `Clase.__mro__` o `Clase.mro()`.

Sección 2. Comprehensions

Ejercicio 2.1 · Precios con IVA

Enunciado: dada la lista `precios = [8.50, 12.00, 3.20, 45.90, 19.99, 7.00]`, usa **una sola list comprehension** para obtener los precios con IVA (21 %), solo de los productos que cuestan **más de 10 €** sin IVA. Redondea a 2 decimales.

Salida esperada: `[14.52, 55.54, 24.19]`

✅ Mi solución

```
precios = [8.50, 12.00, 3.20, 45.90, 19.99, 7.00]

productos_IVA = [round(num * 1.21, 2) for num in precios if num > 10]
print(productos_IVA)
```

Explicación: el `if` al final **filtra** (solo pasan los mayores de 10) y la expresión del principio **transforma** (precio × 1.21 redondeado).

Ejercicio 2.2 · Clasificar paquetes

Enunciado: con `pesos = [0.5, 3.2, 12.0, 1.1, 25.4, 7.8]`, crea con una list comprehension una lista con `"LIGERO"` (menos de 5 kg) o `"PESADO"`. Muestra cuántos son pesados.

Salida esperada

```
['LIGERO', 'LIGERO', 'PESADO', 'LIGERO', 'PESADO', 'PESADO']
Paquetes pesados: 3
```

✅ Mi solución

```
pesos = [0.5, 3.2, 12.0, 1.1, 25.4, 7.8]

medicion_peso = ["LIGERO" if num < 5 else "PESADO" for num in pesos]
```

```
print(medicion_peso)
print("Paquetes pesados: " + str(medicion_peso.count("PESADO")))
```

Explicación: aquí el `if/else` va **al principio** porque se genera un valor distinto para cada elemento (no se descarta ninguno). `count()` cuenta las apariciones de un valor en la lista.

Ejercicio 2.3 · Ciudades sin repetir

Enunciado: con `ciudades = ["Madrid", " madrid", "SEVILLA", "Valencia ", "sevilla", "Bilbao", "MADRID"]` y **una set comprehension**, obtén las ciudades distintas, sin espacios sobrantes y con solo la primera letra en mayúscula. Muestra el resultado **ordenado alfabéticamente** y el número de ciudades distintas.

Salida esperada

```
['Bilbao', 'Madrid', 'Sevilla', 'Valencia']
Ciudades distintas: 4
```

⚠️ Versión corregida

```
ciudades = ["Madrid", " madrid", "SEVILLA", "Valencia ", "sevilla", "Bilbao", "MADRID"]

ciudades_distintas = {ciudad.strip().capitalize() for ciudad in ciudades}

print(sorted(ciudades_distintas))
print("Ciudades distintas:", len(ciudades_distintas))
```

Qué fallaba: imprimía el `set` directamente. Un set **no tiene orden**, así que hay que usar `sorted(...)` al mostrarlo (devuelve una lista ordenada). Además, el texto de salida no coincidía con el pedido.

Explicación: `strip()` quita espacios, `capitalize()` pone la primera en mayúscula y el resto en minúscula, y el **set** elimina los duplicados automáticamente.

> 🔁 **Refuerzo:** `{expr for x in lista}` → **set**. `{k: v for ...}` → **diccionario**. `[expr for ...]` → **lista**. Un set no se puede ordenar: `sorted()` lo convierte en lista ordenada.

Ejercicio 2.4 · Butacas libres en un cine

Enunciado: con `butacas_libres = {"Sala 1": 0, "Sala 2": 34, "Sala 3": 6, "Sala 4": 0, "Sala 5": 120}` crea con comprehensions:

1. Una **lista** con las salas completas (0 libres).
2. Un diccionario sala → estado: `"COMPLETA"` (0 libres), `"ÚLTIMAS PLAZAS"` (menos de 10), `"DISPONIBLE"` (resto).

3. Un diccionario solo con las salas **no** completas, con el **porcentaje de ocupación** (todas tienen 150 butacas), a 1 decimal.

Salida esperada

```
['Sala 1', 'Sala 4']
{'Sala 1': 'COMPLETA', 'Sala 2': 'DISPONIBLE', 'Sala 3': 'ÚLTIMAS PLAZAS', 'Sala 4': 'COMPLETA', 'Sala 5': 'DISPONIBLE'}
{'Sala 2': 77.3, 'Sala 3': 96.0, 'Sala 5': 20.0}
```

⚠️ **Versión corregida**

```
butacas_libres = {"Sala 1": 0, "Sala 2": 34, "Sala 3": 6, "Sala 4": 0, "Sala 5": 120}

# 1. Salas completas
completas = [sala for sala, libres in butacas_libres.items() if libres == 0]
print(completas)

# 2. Estado de cada sala
estados = {sala: "COMPLETA" if libres == 0
                 else "ÚLTIMAS PLAZAS" if libres < 10
                 else "DISPONIBLE"
           for sala, libres in butacas_libres.items()}
print(estados)

# 3. Porcentaje de ocupación de las no completas
ocupacion = {sala: round((150 - libres) / 150 * 100, 1)
             for sala, libres in butacas_libres.items() if libres != 0}
print(ocupacion)
```

Qué fallaba

- Escribí `"ULTIMAS PLAZAS"` **sin tilde**: el texto debe ser exactamente `"ÚLTIMAS PLAZAS"`.
- El apartado 3 estaba bien en resultado (`100 - libres * (100/150)` da lo mismo), pero necesitaba **dos pasos**. Se puede hacer con una sola comprehension con `if` al final y la fórmula directa: ocupadas = 150 − libres.

Explicación: en el apartado 2 se encadenan condicionales `A if cond1 else B if cond2 else C`. En un diccionario, el valor puede ser una expresión condicional completa.

Ejercicio 2.5 · Unir las cestas de la compra

Enunciado: con `cestas = [["pan", "leche"], ["huevos"], [], ["leche", "café", "pan"]]`:

1. Con una list comprehension **con dos `for`**, crea una lista única con todos los productos.
2. Con una dictionary comprehension, crea un diccionario con cuántas veces aparece cada producto (pista: `count()`).

Salida esperada

```
['pan', 'leche', 'huevos', 'leche', 'café', 'pan']
{'pan': 2, 'leche': 2, 'huevos': 1, 'café': 1}
```

✅ **Mi solución**

```
cestas = [["pan", "leche"], ["huevos"], [], ["leche", "café", "pan"]]

productos_totales = [producto for cesta in cestas for producto in cesta]
print(productos_totales)

conteo = {producto: productos_totales.count(producto) for producto in
productos_totales}
print(conteo)
```

Explicación

- Con dos `for`, el **orden de lectura** es el mismo que en los bucles anidados: primero `for cesta in cestas` (el externo), después `for producto in cesta` (el interno).
- En el diccionario, las claves repetidas se **sobrescriben** con el mismo valor, por eso no aparecen duplicadas.

Sección 3. Programación funcional

Datos de la empresa de taxis, comunes a toda la sección:

```
from functools import reduce

carreras = [
    {"id": 1, "pasajero": "Raúl",    "km": 12.0, "importe": 18.5, "tarjeta":
True},
    {"id": 2, "pasajero": "Sofía",   "km": 3.5,  "importe": 7.0,  "tarjeta":
False},
    {"id": 3, "pasajero": "Iván",    "km": 28.0, "importe": 42.0, "tarjeta":
True},
    {"id": 4, "pasajero": "Claudia", "km": 8.0,  "importe": 13.5, "tarjeta":
True},
    {"id": 5, "pasajero": "Hugo",    "km": 15.0, "importe": 24.0, "tarjeta":
False},
]
```

> El campo `tarjeta` indica si el pasajero pagó con tarjeta (`True`) o en efectivo (`False`).

Ejercicio 3.1 · Funciones lambda

Enunciado

1. Lambda `con_suplemento` que sume 2.50 € a un importe.
2. Lambda `es_larga` que devuelva `True` si los km son 10 o más.
3. Con `sorted()` y una lambda, ordena las carreras de más a menos km y muestra solo el nombre.

Salida esperada

```
21.0
False
['Iván', 'Hugo', 'Raúl', 'Claudia', 'Sofía']
```

⚠️ **Versión corregida**

```
con_suplemento = lambda importe: importe + 2.50
print(con_suplemento(18.5))

es_larga = lambda km: km >= 10
print(es_larga(3.5))

nombres = [c["pasajero"] for c in sorted(carreras, key=lambda c: c["km"],
reverse=True)]
print(nombres)
```

Qué mejorar: escribí `lambda km: True if km >= 10 else False`. Funciona, pero es redundante: `km >= 10` **ya es** un booleano.

> 🔁 **Refuerzo:** nunca escribas `True if cond else False`; basta con `cond`. Igual con `== True` o `== False`.

Explicación: `sorted(lista, key=función)` ordena según el valor que devuelve la función para cada elemento; `reverse=True` invierte el orden (de mayor a menor).

Ejercicio 3.2 · `map()`

Enunciado: con `map()` y una lambda, obtén (1) la lista de km y (2) una lista de textos `"Carrera 1: 18.5 €"`.

Salida esperada

```
[12.0, 3.5, 28.0, 8.0, 15.0]
['Carrera 1: 18.5 €', 'Carrera 2: 7.0 €', 'Carrera 3: 42.0 €', 'Carrera 4: 13.5
€', 'Carrera 5: 24.0 €']
```

⚠️ **Versión corregida**

```
kilometros = list(map(lambda c: c["km"], carreras))
print(kilometros)

textos = list(map(lambda c: f"Carrera {c['id']}: {c['importe']} €", carreras))
print(textos)
```

Qué fallaba

- Dejé un **espacio de más** al final del texto (`" € "`), así que cada cadena no coincidía con la esperada.
- Usé comillas dobles dentro de un f-string con comillas dobles (`f"...{c["id"]}..."`). Eso **solo funciona desde Python 3.12**; en versiones anteriores da `SyntaxError`. Lo seguro es usar comillas **simples** dentro (`c['id']`).

Explicación: `map(función, iterable)` aplica la función a cada elemento y devuelve un **iterador** (perezoso), por eso se envuelve con `list(...)`.

Ejercicio 3.3 · `filter()`

Enunciado: con `filter()` y una lambda, obtén (1) los pasajeros que pagaron **en efectivo** (solo el nombre) y (2) el `id` de las carreras pagadas **con tarjeta y de 10 km o más**.

Salida esperada

```
['Sofía', 'Hugo']
[1, 3]
```

⚠️ Versión corregida

```
efectivo = filter(lambda c: not c["tarjeta"], carreras)
print([c["pasajero"] for c in efectivo])

tarjeta_larga = filter(lambda c: c["tarjeta"] and c["km"] >= 10, carreras)
print([c["id"] for c in tarjeta_larga])
```

Qué mejorar: escribí `x["tarjeta"] == False and x["pasajero"]`. El `and x["pasajero"]` sobra (un nombre no vacío siempre es "verdadero") y `== False` se escribe mejor como `not`.

Explicación: `filter(función, iterable)` se queda con los elementos para los que la función devuelve `True`. Igual que `map`, devuelve un iterador.

Ejercicio 3.4 · `reduce()`

Enunciado: con `reduce` (de `functools`) calcula (1) la recaudación total, (2) la carrera más larga (nombre) y (3) los km recorridos en carreras pagadas con tarjeta, combinando `filter()` y `reduce()`.

Salida esperada

```
Recaudación total: 105.0 €
Carrera más larga: Iván
Km pagados con tarjeta: 48.0
```

⚠️ **Versión con el formato pedido**

```
from functools import reduce

recaudacion = reduce(lambda acc, c: acc + c["importe"], carreras, 0)
print(f"Recaudación total: {recaudacion} €")

mas_larga = reduce(lambda a, b: a if a["km"] >= b["km"] else b, carreras)
print(f"Carrera más larga: {mas_larga['pasajero']}")

con_tarjeta = filter(lambda c: c["tarjeta"], carreras)
km_tarjeta = reduce(lambda acc, c: acc + c["km"], con_tarjeta, 0)
print(f"Km pagados con tarjeta: {km_tarjeta}")
```

Qué fallaba: la lógica era correcta, pero mis textos de salida no coincidían con los del enunciado ("La recaudación total son de : ..."). En un examen conviene copiar **exactamente** el formato pedido.

Explicación

- `reduce(función, iterable, inicial)` va **acumulando**: la función recibe `(acumulado, elemento)` y devuelve el nuevo acumulado.
- Cuando se **suma un campo de diccionarios** hay que dar un valor inicial (`0`); si no, `reduce` empezaría con el primer diccionario entero como acumulador y fallaría.
- Para encontrar el **máximo** no hace falta valor inicial: el acumulador es "la mejor carrera hasta ahora".

Ejercicio 3.5 · Funciones que reciben funciones

Enunciado: escribe `transformar_importes(carreras, operacion)` que reciba la lista de carreras y **una función**, y devuelva una lista con el resultado de aplicarla al importe de cada carrera. Sin `map()`: con list comprehension. Pruébala con una función normal `a_dolares(x)` (cambio 1.08, 2 decimales) y una lambda con 10 % de propina.

Salida esperada

```
[19.98, 7.56, 45.36, 14.58, 25.92]
[20.35, 7.7, 46.2, 14.85, 26.4]
```

✅ **Mi solución** (renombrada a `transformar_importes`, como pide el enunciado)

```python
def transformar_importes(carreras, operacion):
    return [operacion(c["importe"]) for c in carreras]

def a_dolares(x):
    return round(x * 1.08, 2)

print(transformar_importes(carreras, a_dolares))
print(transformar_importes(carreras, lambda x: round(x * 1.10, 2)))
```

Explicación: en Python las funciones son **objetos**: se pueden pasar como argumento. Dentro de `transformar_importes`, `operacion` es simplemente una función que se llama con `operacion(...)`. Al pasar `a_dolares` **no** se pone paréntesis: se pasa la función, no su resultado.

Ejercicio 3.6 · Funciones que devuelven funciones

Enunciado: escribe `crear_tarifa(precio_km)` que devuelva otra función que reciba unos km y devuelva el precio del trayecto redondeado a 2 decimales.

Salida esperada

```
11.0
13.5
[16.2, 4.73, 37.8, 10.8, 20.25]
```

✅ Mi solución

```python
def crear_tarifa(precio_km):
    def precio_trayecto(km):
        return round(km * precio_km, 2)
    return precio_trayecto

tarifa_diurna = crear_tarifa(1.10)
tarifa_nocturna = crear_tarifa(1.35)

print(tarifa_diurna(10))
print(tarifa_nocturna(10))
print([tarifa_nocturna(c["km"]) for c in carreras])
```

Explicación: es una **clausura** (*closure*): `precio_trayecto` recuerda el valor de `precio_km` aunque `crear_tarifa` ya haya terminado. Cada llamada a `crear_tarifa` fabrica una función distinta con su propio precio.

> 🔁 **Refuerzo:** nombres de funciones y variables en **snake_case** (`precio_trayecto`), no en camelCase (`precioTrayecto`).

🔁 Resumen funcional

Herramienta	Qué hace	Devuelve
`map(f, it)`	Transforma cada elemento	iterador
`filter(f, it)`	Se queda con los que cumplen `f`	iterador
`reduce(f, it, ini)`	Acumula todo en un único valor	un valor
`sorted(it, key=f)`	Ordena según `f`	lista

Sección 4. Recursión

Sin bucles (`for`/`while`) salvo que el enunciado lo permita. Toda función recursiva necesita:

1. **Caso base:** cuándo parar.
2. **Caso recursivo:** resolver un trocito y delegar el resto en una llamada a **sí misma** con un problema más pequeño.

Ejercicio 4.1 · Sumar los dígitos de un código

Enunciado: `suma_digitos(n)` devuelve la suma de las cifras de un entero positivo. Pista: `n % 10` es la última cifra y `n // 10` la quita.

Salida esperada

```
19
7
```

✅ Mi solución

```python
def suma_digitos(n):
    if n == 0:
        return 0
    cifra = n % 10
    return cifra + suma_digitos(n // 10)

print(suma_digitos(4528))
print(suma_digitos(7))
```

Desarrollo

```
suma_digitos(4528) = 8 + suma_digitos(452)
                   = 8 + 2 + suma_digitos(45)
                   = 8 + 2 + 5 + suma_digitos(4)
```

```
                    = 8 + 2 + 5 + 4 + suma_digitos(0)
                    = 8 + 2 + 5 + 4 + 0 = 19
```

Ejercicio 4.2 · Total de una lista de gastos

Enunciado: `total_gastos(gastos)` suma los valores de una lista. Idea: *el total es el primer elemento más el total del resto*. Caso base: lista vacía.

Salida esperada

```
100.0
0
```

✅ Mi solución

```python
def total_gastos(gastos):
    if gastos == []:
        return 0
    return gastos[0] + total_gastos(gastos[1:])

print(total_gastos([12.5, 30, 7.5, 50]))
print(total_gastos([]))
```

Explicación: `gastos[0]` es el primero y `gastos[1:]` es "el resto" (la lista sin el primero), cada vez más corta hasta quedar vacía.

Ejercicio 4.3 · ¿Es un palíndromo?

Enunciado: `es_palindromo(texto)` devuelve `True` o `False`. Pista: compara el primer y el último carácter y, si coinciden, vuelve a llamar con el texto sin esos dos.

Salida esperada

```
True
True
False
```

✅ Mi solución

```python
def es_palindromo(texto):
    if len(texto) <= 1:
        return True
    if texto[0] == texto[-1]:
```

```
        return es_palindromo(texto[1:-1])
    return False

print(es_palindromo("reconocer"))
print(es_palindromo("radar"))
print(es_palindromo("python"))
```

Explicación: el caso base es `len(texto) <= 1` (vacío o un solo carácter siempre es palíndromo). Esto cubre palabras de longitud par e impar. `texto[1:-1]` quita el primer y el último carácter.

Ejercicio 4.4 · Cajas dentro de cajas

Enunciado: cada caja es una lista y cada artículo un número (su peso en kg). Con `envio = [2, [3, 1], [[4, 5], 2], [], 6]`:

1. `peso_total(caja)`: kilos de todo el contenido.
2. `contar_articulos(caja)`: cuántos artículos hay (las cajas no cuentan).

Aquí **sí se permite un `for`** para recorrer el contenido de cada caja.

Salida esperada

```
Peso total: 23 kg
Número de artículos: 7
```

✅ **Mi solución** (con el formato de salida del enunciado)

```
envio = [2, [3, 1], [[4, 5], 2], [], 6]

def peso_total(caja):
    total = 0
    for elemento in caja:
        if isinstance(elemento, list):
            total += peso_total(elemento)
        else:
            total += elemento
    return total

def contar_articulos(caja):
    total = 0
    for elemento in caja:
        if isinstance(elemento, list):
            total += contar_articulos(elemento)
        else:
            total += 1
    return total
```

```
print(f"Peso total: {peso_total(envio)} kg")
print(f"Número de artículos: {contar_articulos(envio)}")
```

Explicación: el caso base está **implícito**: cuando la caja está vacía, el `for` no itera y devuelve 0. Cada elemento que es una lista se resuelve con una llamada recursiva (caja dentro de caja); si es un número, se suma directamente.

> 🔁 **Refuerzo:** `isinstance(x, list)` distingue una caja de un artículo. Es el patrón típico para recorrer estructuras **anidadas**.

Ejercicio 4.5 · Iterativo frente a recursivo

Enunciado: una cuenta da un 2 % de interés anual. Escribe `ahorro_iterativo(capital, anios)` (con bucle) y `ahorro_recursivo(capital, anios)` (sin bucles). Redondea a 2 decimales al mostrar.

Salida esperada

```
1104.08
1104.08
```

⚠️ Versión corregida

```
def ahorro_iterativo(capital, anios):
    for _ in range(anios):
        capital = capital * 1.02
    return capital

def ahorro_recursivo(capital, anios):
    if anios == 0:
        return capital
    return ahorro_recursivo(capital * 1.02, anios - 1)

print(round(ahorro_iterativo(1000, 5), 2))
print(round(ahorro_recursivo(1000, 5), 2))
```

Qué fallaba (importante): en la versión recursiva, la última línea llamaba a `ahorro_iterativo` en lugar de a `ahorro_recursivo`. El resultado salía igual, pero la función **no era recursiva**: una función recursiva tiene que llamarse **a sí misma**.

Pregunta: ¿qué pasaría con la versión recursiva si se llamara con `anios = 5000`? Daría un `RecursionError` (*maximum recursion depth exceeded*): Python limita la profundidad de recursión (por defecto, unas 1000 llamadas) y se necesitarían 5000 llamadas apiladas. La versión iterativa no tiene ese límite.

> 🔁 **Refuerzo:** la recursión es elegante pero consume **una llamada apilada por paso**. Para repeticiones muy largas es mejor un bucle.

Sección 5. Clases y objetos

Ejercicio 5.1 · Artículos de una tienda de deportes

Enunciado: crea la clase `Articulo` con:

- Atributos `nombre`, `precio` y `stock` (por defecto 0).
- `valor_inventario()`: precio × stock.
- `vender(cantidad)`: si hay stock suficiente lo resta y devuelve `True`; si no, no cambia nada y devuelve `False`.
- `reponer(cantidad)`: suma unidades al stock.
- `__str__`: texto como `Balón de fútbol | 25.0 € | 10 uds.`

Salida esperada

```
Balón de fútbol | 25.0 € | 10 uds.
Cinta de pádel | 9.5 € | 0 uds.
True
False
Balón de fútbol | 25.0 € | 7 uds.
Cinta de pádel | 9.5 € | 20 uds.
175.0 190.0
```

⚠️ Versión corregida

```
class Articulo:
    def __init__(self, nombre, precio, stock=0):
        self.nombre = nombre
        self.precio = precio
        self.stock = stock

    def valor_inventario(self):
        return self.precio * self.stock

    def vender(self, cantidad):
        if self.stock >= cantidad:
            self.stock -= cantidad
            return True
        return False

    def reponer(self, cantidad):
        self.stock += cantidad

    def __str__(self):
        return f"{self.nombre} | {self.precio} € | {self.stock} uds."


a1 = Articulo("Balón de fútbol", 25.0, 10)
a2 = Articulo("Cinta de pádel", 9.5)
```

```
print(a1)
print(a2)
print(a1.vender(3))
print(a2.vender(1))
a2.reponer(20)
print(a1)
print(a2)
print(a1.valor_inventario(), a2.valor_inventario())
```

Qué fallaba: mi `__str__` devolvía `nombre | precio | stock`, sin el símbolo `€` ni `uds.`. La lógica de `vender`, `reponer` y `valor_inventario` estaba bien.

Explicación

- `stock=0` es un **parámetro por defecto**: si no se indica, vale 0.
- `vender` **muta** el objeto (`self.stock -= cantidad`) y además **devuelve** un booleano para informar del resultado.
- `__str__` es lo que usa `print(objeto)`.

Ejercicio 5.2 · Cuentas de un banco

Enunciado: crea la clase `Cuenta` con:

- Atributo de clase `total_cuentas` (cuenta cuántas cuentas se han creado) y `COMISION_RETIRADA = 1.5`.
- Atributos de instancia `titular` y `_saldo` (0.0 por defecto).
- `ingresar(importe)`: suma el importe al saldo **solo si es positivo**.
- `retirar(importe)`: resta el importe **más la comisión**, solo si hay saldo suficiente para ambos; devuelve `True`/`False`.
- `consultar_saldo()`.
- `cuentas_creadas()`: `@classmethod` que devuelve `total_cuentas`.
- `__repr__`: texto como `Cuenta('Laura', 150.0)`.

Salida esperada

```
True
False
Cuenta('Laura', 100.0) Cuenta('Diego', 0.0)
Cuentas creadas: 2
```

⚠️ Versión corregida

```
class Cuenta:
    total_cuentas = 0
    COMISION_RETIRADA = 1.5

    def __init__(self, titular, saldo=0.0):
```

```
        self.titular = titular
        self._saldo = saldo
        Cuenta.total_cuentas += 1

    def ingresar(self, importe):
        if importe > 0:
            self._saldo += importe

    def retirar(self, importe):
        total = importe + self.COMISION_RETIRADA
        if total <= self._saldo:
            self._saldo -= total
            return True
        return False

    def consultar_saldo(self):
        return self._saldo

    @classmethod
    def cuentas_creadas(cls):
        return cls.total_cuentas

    def __repr__(self):
        return f"Cuenta({self.titular!r}, {self._saldo})"


c1 = Cuenta("Laura", 100.0)
c2 = Cuenta("Diego")

c1.ingresar(50)
c1.ingresar(-20)
print(c1.retirar(48.5))
print(c2.retirar(10))
print(c1, c2)
print("Cuentas creadas:", Cuenta.cuentas_creadas())
```

Qué fallaba (este era el ejercicio con más errores)

Mi código	Problema	Corrección
`total_cuentas = []` y `append(self)`	El enunciado pide un **contador**	`total_cuentas = 0` y `Cuenta.total_cuentas += 1`
`ingresar` hacía `return self._saldo + importe`	**Calculaba pero no guardaba** nada: el saldo nunca cambiaba	`self._saldo += importe`
`if importe > self._saldo and ...`	Condición **invertida**: solo retiraba si pedía *más* de lo que tenía	`if total <= self._saldo`

Mi código	Problema	Corrección
`comision = self._saldo + COMISION` y `importe - comision`	La comisión se **sumaba al saldo** y la resta no se guardaba en ningún sitio	`total = importe + COMISION` y `self._saldo -= total`
`f"Cuenta({self.titular}, {self._saldo})"`	Faltaban las comillas del titular	`{self.titular!r}`

Explicación del recorrido

1. `c1` empieza con 100.0. `ingresar(50)` → 150.0. `ingresar(-20)` se ignora (no es positivo).
2. `c1.retirar(48.5)`: total = 48.5 + 1.5 = 50.0 ≤ 150.0 → el saldo pasa a 100.0 y devuelve `True`.
3. `c2.retirar(10)`: total = 11.5 > 0.0 → devuelve `False`.
4. `print(c1, c2)` usa `__repr__` (no hay `__str__`).

> 🔁 **Refuerzo (muy importante):** un método que **modifica** el estado debe hacer `self.x = ...` / `self.x += ...`. Hacer `return self.x + algo` solo **calcula** un valor y lo devuelve; el atributo no cambia.
>
> 🔁 `!r` dentro de un f-string llama a `repr()`: por eso `'Laura'` sale **con comillas**.
>
> 🔁 **Atributo de clase vs instancia:** para modificar uno compartido hay que escribir `Cuenta.total_cuentas` (o `cls.total_cuentas` en un `@classmethod`).

Ejercicio 5.3 · Carrito de la compra (composición)

Enunciado: con la clase `Articulo` del 5.1, crea `Carrito`, que **contiene** una lista de líneas `(articulo, cantidad)`:

- `agregar(articulo, cantidad)`.
- `total()`: importe total.
- `resumen()`: cada línea con formato `3 x Balón de fútbol = 75.0 €` y al final el total.

Salida esperada

```
3 x Balón de fútbol = 75.0 €
2 x Cinta de pádel = 19.0 €
1 x Botella deportiva = 15.0 €
TOTAL: 109.0 €
```

⚠️ Versión corregida

```python
class Carrito:
    def __init__(self):
        self.lineas = []

    def agregar(self, articulo, cantidad):
```

```
        self.lineas.append((articulo, cantidad))

    def total(self):
        total = 0
        for articulo, cantidad in self.lineas:
            total += articulo.precio * cantidad
        return total

    def resumen(self):
        for articulo, cantidad in self.lineas:
            importe = articulo.precio * cantidad
            print(f"{cantidad} x {articulo.nombre} = {importe} €")
        print(f"TOTAL: {self.total()} €")


carrito = Carrito()
carrito.agregar(Articulo("Balón de fútbol", 25.0, 10), 3)
carrito.agregar(Articulo("Cinta de pádel", 9.5, 20), 2)
carrito.agregar(Articulo("Botella deportiva", 15.0, 5), 1)
carrito.resumen()
```

Qué fallaba: imprimía `Total:` en lugar de `TOTAL:`. La lógica estaba bien.

Explicación: esto es **composición** ("tiene un"): `Carrito` **contiene** objetos `Articulo` dentro de una lista, en lugar de heredar de ellos. Cada línea es una **tupla** que se desempaqueta en el `for`.

> 🔁 **Herencia vs composición:** herencia = "es un" (`Moto` **es un** `Vehiculo`). Composición = "tiene un" (`Carrito` **tiene** artículos).

Ejercicio 5.4 · Lista de objetos

Enunciado: con la clase `Articulo` y este catálogo, resuelve cada apartado en **una línea**:

```
catalogo = [
    Articulo("Esterilla", 30.0, 12),
    Articulo("Comba", 12.0, 0),
    Articulo("Bicicleta estática", 180.0, 3),
    Articulo("Mancuernas", 45.0, 0),
]
```

1. Nombres de los artículos sin stock.
2. Diccionario `nombre → valor_inventario()`.
3. Catálogo ordenado por precio de menor a mayor, mostrando cada artículo con `print()`.

Salida esperada

```
['Comba', 'Mancuernas']
{'Esterilla': 360.0, 'Comba': 0.0, 'Bicicleta estática': 540.0, 'Mancuernas': 0.0}
```

```
Comba | 12.0 € | 0 uds.
Esterilla | 30.0 € | 12 uds.
Mancuernas | 45.0 € | 0 uds.
Bicicleta estática | 180.0 € | 3 uds.
```

✅ **Mi solución** (necesita el `__str__` corregido del 5.1 para que salga el formato esperado)

```python
sin_stock = [a.nombre for a in catalogo if a.stock == 0]
print(sin_stock)

valores = {a.nombre: a.valor_inventario() for a in catalogo}
print(valores)

for a in sorted(catalogo, key=lambda a: a.precio):
    print(a)
```

Explicación: `sorted(..., key=lambda a: a.precio)` ordena objetos por un **atributo**. Es el mismo patrón que con diccionarios, pero accediendo con `.atributo` en vez de `["clave"]`.

Sección 6. Sobrecarga de operadores y conversión de tipos

La idea clave

Cuando escribes `a + b`, Python **no suma directamente**: llama al método especial `a.__add__(b)`. Esos métodos con doble guion bajo (*dunder methods* o **métodos mágicos**) son los que definen cómo se comporta tu clase con los operadores y funciones de Python. **Sobrecargar un operador** = definir ese método en tu clase.

Tabla de métodos especiales

Operadores aritméticos

Expresión	Método que se llama	Notas
`a + b`	`a.__add__(b)`	Debe devolver un objeto **nuevo**
`a - b`	`a.__sub__(b)`	
`a * b`	`a.__mul__(b)`	
`a / b`	`a.__truediv__(b)`	
`a // b`	`a.__floordiv__(b)`	
`a % b`	`a.__mod__(b)`	
`a ** b`	`a.__pow__(b)`	
`-a`	`a.__neg__()`	

Expresión	Método que se llama	Notas
`5 + a`	`a.__radd__(5)`	Se usa cuando el **izquierdo** no sabe sumar con tu clase (ver `sum()` más abajo)
`a += b`	`a.__iadd__(b)`	Debe **devolver** `self`

Comparaciones

Expresión	Método
`a == b`	`a.__eq__(b)`
`a != b`	`a.__ne__(b)` (por defecto es lo contrario de `__eq__`)
`a < b`	`a.__lt__(b)`
`a <= b`	`a.__le__(b)`
`a > b`	`a.__gt__(b)`
`a >= b`	`a.__ge__(b)`

Conversión a otros tipos y representación

Expresión	Método	Debe devolver
`str(a)` / `print(a)`	`a.__str__()`	un `str` (texto "bonito" para el usuario)
`repr(a)` / objetos dentro de una lista	`a.__repr__()`	un `str` (texto "técnico", idealmente sin ambigüedad)
`float(a)`	`a.__float__()`	un `float`
`int(a)`	`a.__int__()`	un `int`
`bool(a)` / `if a:` / `not a`	`a.__bool__()`	`True` o `False`

Comportamiento de contenedor

Expresión	Método
`len(a)`	`a.__len__()`
`x in a`	`a.__contains__(x)`
`a[i]`	`a.__getitem__(i)`
`a[i] = v`	`a.__setitem__(i, v)`
`for x in a`	`a.__iter__()`
`a(...)`	`a.__call__(...)`

Reglas que hay que saber

1. `__add__` **devuelve un objeto nuevo**; `__iadd__` (`+=`) modifica y **debe terminar con** `return self`. Si no devuelve nada, tras `viaje += "x"` la variable `viaje` pasaría a valer `None`.
2. `__str__` **vs** `__repr__`**:** `print(obj)` usa `__str__`; si no existe, usa `__repr__`. Los objetos **dentro de una lista** se muestran siempre con `__repr__`.
3. `__lt__` **es el que usa** `sorted()` y `min()`/`max()`. Con definir `__eq__` y `__lt__` se pueden ordenar objetos.
4. `__bool__`**:** si no existe, Python usa `__len__` (0 = falso). Si no existe ninguno, el objeto es siempre `True`.
5. `__eq__` **y** `__hash__`**:** si defines `__eq__` sin `__hash__`, el objeto deja de poder usarse como clave de diccionario o dentro de un set.
6. **Los métodos especiales no se llaman a mano:** se escribe `a + b`, no `a.__add__(b)`.
7. `sum()` **empieza en** `0`**:** `sum([Importe(1), Importe(2)])` calcula `0 + Importe(1)`, que necesita `__radd__`. Solución: `sum(lista, Importe(0))`.

Métodos normales, de clase y estáticos (aparecen en 5.2 y 6.3)

Tipo	Decorador	Primer parámetro	Accede a...
De instancia	(ninguno)	`self`	la instancia y la clase
De clase	`@classmethod`	`cls`	la clase (no a una instancia concreta)
Estático	`@staticmethod`	ninguno	nada: es una función "guardada" dentro de la clase

---

Ejercicio 6.1 · Sumar y comparar importes

Enunciado: crea una clase `Importe` que guarde una `cantidad` en euros y haga funcionar:

- `+` entre dos importes → devuelve un **nuevo** `Importe`.
- `==` y `<` → comparan las cantidades.
- `str()` → importe con 2 decimales y el símbolo `€`.
- `float()` → la cantidad como decimal.

Salida esperada

```
25.00 €
True
False
39.98
```

✅ Mi solución

```
class Importe:
    def __init__(self, cantidad):
        self.cantidad = cantidad
```

```
    def __add__(self, otro):
        return Importe(self.cantidad + otro.cantidad)

    def __eq__(self, otro):
        return round(self.cantidad, 2) == round(otro.cantidad, 2)

    def __lt__(self, otro):
        return self.cantidad < otro.cantidad

    def __str__(self):
        return f"{self.cantidad:.2f} €"

    def __float__(self):
        return float(self.cantidad)


a = Importe(19.99)
b = Importe(5.01)
c = Importe(25)

total = a + b
print(total)
print(total == c)
print(a < b)
print(float(a) * 2)
```

Qué método se ejecuta en cada línea

Línea	Python ejecuta	Resultado
`total = a + b`	`a.__add__(b)` → crea `Importe(25.0)`	objeto nuevo
`print(total)`	`total.__str__()`	`25.00 €`
`total == c`	`total.__eq__(c)`	`True`
`a < b`	`a.__lt__(b)` → `19.99 < 5.01`	`False`
`float(a) * 2`	`a.__float__()` → `19.99`, y luego `19.99 * 2`	`39.98`

Explicación

- `__eq__` **redondea** a 2 decimales porque los `float` arrastran errores de precisión (por ejemplo, `0.1 + 0.2` da `0.30000000000000004` y no `0.3`). Así dos importes "iguales" se consideran iguales.
- `__add__` crea un `Importe` **nuevo**: no modifica `a` ni `b`.
- `__float__` permite usar el objeto en cálculos numéricos con `float(objeto)`.

Ejercicio 6.2 · Lista de reproducción

Enunciado: crea `ListaReproduccion` con un `nombre` y una lista interna de canciones. Haz que funcionen `len(lista)`, `"titulo" in lista`, `lista += "titulo"` (añade y devuelve la propia lista) y `str(lista)` (`Viaje (3 canciones)`).

Salida esperada

```
Viaje (3 canciones)
3
True
False
```

✅ Mi solución

```
class ListaReproduccion:
    def __init__(self, nombre):
        self.nombre = nombre
        self.canciones = []

    def __len__(self):
        return len(self.canciones)

    def __contains__(self, titulo):
        return titulo in self.canciones

    def __iadd__(self, titulo):
        self.canciones.append(titulo)
        return self

    def __str__(self):
        return f"{self.nombre} ({len(self)} canciones)"


viaje = ListaReproduccion("Viaje")
viaje += "Canción A"
viaje += "Canción B"
viaje += "Canción C"

print(viaje)
print(len(viaje))
print("Canción B" in viaje)
print("Canción Z" in viaje)
```

Qué método se ejecuta en cada línea

Línea	Python ejecuta
`viaje += "Canción A"`	`viaje = viaje.__iadd__("Canción A")`
`print(viaje)`	`viaje.__str__()`
`len(viaje)`	`viaje.__len__()`
`"Canción B" in viaje`	`viaje.__contains__("Canción B")`

Explicación

- `__iadd__` **modifica** la lista interna y **devuelve** `self`. Ese `return self` es imprescindible: la expresión `viaje += x` equivale a `viaje = viaje.__iadd__(x)`, así que si no devolviera nada, `viaje` valdría `None`.
- Dentro de `__str__` uso `len(self)`, que a su vez llama a `__len__`: los métodos especiales se pueden reutilizar entre sí.

Ejercicio 6.3 · Precios escritos a mano

Enunciado: un restaurante recibe los precios como texto (`"12,50 €"`).

1. Clase `Precio` que guarde el valor en euros como decimal.
2. **Método estático** `desde_texto(texto)` que reciba `"7,25 €"` y devuelva un `Precio`.
3. `float(precio)` devuelve el valor; `bool(precio)` es `False` si el precio es 0 (plato gratuito); `str(precio)` muestra `7.25 €`.

Salida esperada

```
[12.5, 0.0, 7.25, 18.0]
Total carta: 37.75
Platos gratuitos: 1
7.25 €
```

⚠️ **Versión corregida**

```
class Precio:
    def __init__(self, valor):
        self.valor = valor

    @staticmethod
    def desde_texto(texto):
        texto = texto.replace("€", "").strip()
        texto = texto.replace(",", ".")
        return Precio(float(texto))

    def __float__(self):
        return float(self.valor)

    def __bool__(self):
        return self.valor != 0

    def __str__(self):
        return f"{self.valor:.2f} €"


carta = ["12,50 €", "0,00 €", "7,25 €", "18,00 €"]
precios = [Precio.desde_texto(t) for t in carta]
```

```
print([float(p) for p in precios])
print("Total carta:", sum(float(p) for p in precios))
print("Platos gratuitos:", len([p for p in precios if not p]))
print(precios[2])
```

Qué fallaba

- Mi `__str__` devolvía `"7.25"` sin el símbolo `€`; el enunciado pide `7.25 €`.
- `__bool__` estaba bien pero se puede simplificar: `return self.valor != 0` (la comparación ya es un booleano; no hace falta `if ... return False else return True`).

Qué método se ejecuta en cada línea

Línea	Python ejecuta
`Precio.desde_texto("7,25 €")`	método **estático**: se llama sobre la clase, sin crear un objeto antes
`float(p)`	`p.__float__()`
`if not p`	`p.__bool__()` y se niega el resultado
`print(precios[2])`	`precios[2].__str__()`

Explicación

- `desde_texto` es **estático** porque no necesita ni `self` ni `cls`: solo transforma un texto en un `Precio`. Es un *constructor alternativo* (una forma extra de crear objetos).
- El proceso es: quitar `€` → quitar espacios con `strip()` → cambiar la coma decimal por punto → `float()`.
- `not p` usa `__bool__`: un precio de 0 es "falso", por eso se cuenta como plato gratuito.

> 🔁 **Refuerzo de la sección:** cada vez que el enunciado diga "haz que funcione `X`", busca en la tabla qué método especial lo activa. Y recuerda: el método **devuelve** un valor, no lo imprime.

Sección 7. Herencia, polimorfismo y herencia múltiple

Conceptos clave

- **Herencia:** una subclase (`Moto`) hereda atributos y métodos de la superclase (`Vehiculo`).
- **Sobrescribir (override):** la subclase redefine un método del padre.
- **`super()`:** llama al método del padre (o, con herencia múltiple, al **siguiente en el MRO**).
- **Polimorfismo:** el mismo método (`peaje()`) funciona distinto según el objeto. El código que lo llama no necesita saber de qué clase es.
- **MRO:** orden en que Python busca métodos. Se consulta con `Clase.__mro__`.

Ejercicio 7.1 · Peajes de una autopista

Enunciado

1. Superclase `Vehiculo` con `matricula` y `kilometros`, y `peaje()` que devuelve 0.10 € por km.
2. Subclases que **sobrescriben** `peaje()`:

- `Moto`: paga la mitad.
- `Camion`: atributo extra `ejes`; paga lo mismo que un vehículo normal **más 2 € por eje**. Usa `super()` en el constructor **y** en `peaje()`.

3. Recorre una lista con un `Vehiculo`, una `Moto` y un `Camion` con **un único bucle**, mostrando matrícula y peaje redondeado a 2 decimales.
4. Muestra el total recaudado.

Salida esperada

```
1234-ABC → 10.0 €
5678-DEF → 5.0 €
9012-GHI → 18.0 €
Total recaudado: 33.0 €
```

⚠️ **Versión corregida**

```python
class Vehiculo:
    def __init__(self, matricula, kilometros):
        self.matricula = matricula
        self.kilometros = kilometros

    def peaje(self):
        return 0.10 * self.kilometros


class Moto(Vehiculo):
    def peaje(self):
        return super().peaje() / 2


class Camion(Vehiculo):
    def __init__(self, matricula, kilometros, ejes):
        super().__init__(matricula, kilometros)
        self.ejes = ejes

    def peaje(self):
        return super().peaje() + 2 * self.ejes


vehiculos = [
    Vehiculo("1234-ABC", 100),
    Moto("5678-DEF", 100),
    Camion("9012-GHI", 100, 4),
]

total = 0
for v in vehiculos:
    importe = round(v.peaje(), 2)
    print(f"{v.matricula} → {importe} €")
    total += importe
```

```
print(f"Total recaudado: {round(total, 2)} €")
```

Qué fallaba

- Escribí la fórmula completa (`0.10 * self.kilometros / 2`) en lugar de **reutilizar** el método del padre con `super().peaje()`. El enunciado pedía explícitamente `super()` en `peaje()`.
- Tenía una línea **inalcanzable**: `self.peaje.super()` después de un `return`. Código que viene tras un `return` **nunca se ejecuta**, y además `.super()` no existe (se escribe `super().peaje()`).
- `Moto` no necesita `__init__`: hereda el de `Vehiculo`.
- Para que salga `10.0 €` y no `10.00 €` hay que redondear con `round(x, 2)` en vez de formatear con `:.2f`.

Pregunta: ¿en qué parte del código se aplica el polimorfismo? En el bucle `for v in vehiculos: ... v.peaje()`. Se llama siempre al mismo método, pero Python ejecuta la versión de cada clase (`Vehiculo`, `Moto` o `Camion`) según el objeto real. El bucle no tiene ningún `if` que pregunte por el tipo.

> 🔁 **Refuerzo:** `super().metodo()` **reutiliza** el cálculo del padre; no repitas su fórmula. Después de un `return` no se ejecuta nada.

Ejercicio 7.2 · Suscripciones de una plataforma de series

Enunciado

1. Superclase `Suscripcion` con `usuario` y `cuota_base`, y los métodos `precio_mensual()` (devuelve la cuota base) y `ficha()` (devuelve `"<usuario> (<tipo>): <precio> €"`, con `<tipo>` el nombre de la clase y el precio con 2 decimales; pista: `type(self).__name__`).
2. Subclases: `Familiar` (cuota base + 3 € por perfil extra, recibe `perfiles_extra`), `Anual` (15 % de descuento), `ConAnuncios` (60 % de la cuota).
3. Recorre la lista mostrando la ficha y calcula los ingresos mensuales con `sum()` y una comprehension.

Salida esperada

```
Carlos (Suscripcion): 12.00 €
Lucía (Familiar): 18.00 €
Sergio (Anual): 10.20 €
Nerea (ConAnuncios): 7.20 €
Ingresos mensuales: 47.40 €
```

⚠️ Versión corregida

```
class Suscripcion:
    def __init__(self, usuario, cuota_base):
        self.usuario = usuario
        self.cuota_base = cuota_base

    def precio_mensual(self):
```

```
        return self.cuota_base

    def ficha(self):
        return f"{self.usuario} ({type(self).__name__}):
{self.precio_mensual():.2f} €"


class Familiar(Suscripcion):
    def __init__(self, usuario, cuota_base, perfiles_extra):
        super().__init__(usuario, cuota_base)
        self.perfiles_extra = perfiles_extra

    def precio_mensual(self):
        return self.cuota_base + 3 * self.perfiles_extra


class Anual(Suscripcion):
    def precio_mensual(self):
        return self.cuota_base * 0.85


class ConAnuncios(Suscripcion):
    def precio_mensual(self):
        return self.cuota_base * 0.60


suscripciones = [
    Suscripcion("Carlos", 12),
    Familiar("Lucía", 12, 2),
    Anual("Sergio", 12),
    ConAnuncios("Nerea", 12),
]

for s in suscripciones:
    print(s.ficha())

total = sum(s.precio_mensual() for s in suscripciones)
print(f"Ingresos mensuales: {total:.2f} €")
```

Qué fallaba (importante)

- En `ficha()` usé `self.cuota_base`, no `self.precio_mensual()`. Así **todas** las fichas mostraban 12.00 €, sin aplicar descuentos ni extras. Es justo la gracia del polimorfismo: `ficha()` se escribe **una vez** y llama a `self.precio_mensual()`; Python ejecuta la versión de la clase real del objeto.
- Puse `< >` alrededor del usuario. Los `<...>` del enunciado son **marcadores de posición**, no hay que imprimirlos.

Explicación

- `type(self).__name__` da el nombre de la clase **real** del objeto (`Familiar`, `Anual`...). Por eso `ficha()` muestra el tipo correcto sin tocar las subclases.

- `Familiar` necesita `__init__` propio porque recibe un atributo extra; usa `super().__init__(...)` para reutilizar la inicialización del padre. `Anual` y `ConAnuncios` no lo necesitan.
- `sum(generador)` con una *generator expression* es una comprehension "perezosa": no crea una lista intermedia.

> 🔁 **Refuerzo:** si un método de la superclase necesita un valor que **cambia según la subclase**, llama a `self.metodo()` en vez de usar el atributo directamente.

Ejercicio 7.3 · Impresora multifunción (herencia múltiple)

Enunciado

1. `EquipoOficina` con atributo `modelo` y `encender()` que muestra `"<modelo>: encendido"`.
2. `Impresora(EquipoOficina)` con `imprimir(documento)` → `"Imprimiendo <documento>"`; `Escaner(EquipoOficina)` con `escanear(documento)` → `"Escaneando <documento>"`.
3. `Multifuncion(Impresora, Escaner)` con `fotocopiar(documento)` que reutilice los dos anteriores.
4. Cada clase sobrescribe `descripcion()` usando `super()`, para que `Multifuncion` obtenga la descripción completa.

Salida esperada

```
OfiPro 300: encendido
Escaneando contrato.pdf
Imprimiendo contrato.pdf
Equipo de oficina + Escáner + Impresora + Multifunción
['Multifuncion', 'Impresora', 'Escaner', 'EquipoOficina', 'object']
```

⚠️ Versión corregida

```python
class EquipoOficina:
    def __init__(self, modelo):
        self.modelo = modelo

    def encender(self):
        print(f"{self.modelo}: encendido")

    def descripcion(self):
        return "Equipo de oficina"


class Impresora(EquipoOficina):
    def imprimir(self, documento):
        print(f"Imprimiendo {documento}")

    def descripcion(self):
        return super().descripcion() + " + Impresora"
```

```
class Escaner(EquipoOficina):
    def escanear(self, documento):
        print(f"Escaneando {documento}")

    def descripcion(self):
        return super().descripcion() + " + Escáner"


class Multifuncion(Impresora, Escaner):
    def fotocopiar(self, documento):
        self.escanear(documento)
        self.imprimir(documento)

    def descripcion(self):
        return super().descripcion() + " + Multifunción"


equipo = Multifuncion("OfiPro 300")
equipo.encender()
equipo.fotocopiar("contrato.pdf")
print(equipo.descripcion())
print([clase.__name__ for clase in Multifuncion.__mro__])
```

Qué fallaba: en `EquipoOficina.descripcion()` devolvía `"Equipo de oficina "` **con un espacio al final**; combinado con `" + Impresora"` salía con un doble espacio. Un espacio de más en un `print` hace que la salida no coincida.

Pregunta: ¿por qué la descripción sale en ese orden? Las llamadas a `super()` recorren el **MRO** (`Multifuncion → Impresora → Escaner → EquipoOficina`) **hacia abajo**: `Multifuncion` llama a `Impresora`, que llama a `Escaner` (aunque `Escaner` no sea su padre directo), que llama a `EquipoOficina`. Esta última **devuelve primero** su texto, y al volver cada clase añade el suyo. Por eso el resultado es el MRO leído **al revés**: `Equipo de oficina + Escáner + Impresora + Multifunción`.

> 🔁 **Refuerzo:** en `Impresora`, `super()` no apunta a `EquipoOficina` cuando el objeto es una `Multifuncion`, sino a `Escaner`. `super()` depende del **objeto**, no de la clase donde está escrito el código.

Ejercicio 7.4 · Permisos de usuarios (herencia múltiple con `super()`)

Enunciado: cada tipo de usuario **suma** permisos a los del tipo del que hereda.

Clase	Hereda de	Permiso que añade
`Usuario`	—	`"leer"`
`Editor`	`Usuario`	`"editar"`
`Moderador`	`Usuario`	`"borrar comentarios"`
`Administrador`	`Editor` y `Moderador` (en ese orden)	`"gestionar usuarios"`

Requisitos:

1. Todas reciben solo `nombre`; `Usuario` lo guarda como atributo.
2. El constructor de **cada** clase llama **primero** a `super().__init__(nombre)` y **después** muestra `"Configurando <Clase>"`. `Usuario` no llama a `super()` y solo muestra su mensaje.
3. Cada clase tiene `permisos()`; las subclases **no repiten** los de sus padres: usan `super().permisos()` y añaden el suyo.
4. `Usuario` tiene `puede(accion)`: `True` si la acción está en `self.permisos()`.

Salida esperada

```
Configurando Usuario
Configurando Moderador
Configurando Editor
Configurando Administrador
['leer', 'borrar comentarios', 'editar', 'gestionar usuarios']
True False
['Administrador', 'Editor', 'Moderador', 'Usuario', 'object']
```

⚠️ **Versión corregida**

```python
class Usuario:
    def __init__(self, nombre):
        self.nombre = nombre
        print("Configurando Usuario")

    def permisos(self):
        return ["leer"]

    def puede(self, accion):
        return accion in self.permisos()


class Editor(Usuario):
    def __init__(self, nombre):
        super().__init__(nombre)
        print("Configurando Editor")

    def permisos(self):
        return super().permisos() + ["editar"]


class Moderador(Usuario):
    def __init__(self, nombre):
        super().__init__(nombre)
        print("Configurando Moderador")

    def permisos(self):
        return super().permisos() + ["borrar comentarios"]
```

```
class Administrador(Editor, Moderador):
    def __init__(self, nombre):
        super().__init__(nombre)
        print("Configurando Administrador")

    def permisos(self):
        return super().permisos() + ["gestionar usuarios"]


admin = Administrador("Marina")
print(admin.permisos())
print(admin.puede("borrar comentarios"), admin.puede("borrar la base de datos"))
print([clase.__name__ for clase in Administrador.__mro__])
```

Qué fallaba: devolvía `["Leer"]` con **mayúscula**; el enunciado y la salida esperada usan `"leer"` en minúscula, y con `puede("leer")` habría dado `False`. (Además, `print(f"...")` sin ninguna llave `{}` no necesita la `f`.)

Por qué sale ese orden

El MRO es `Administrador → Editor → Moderador → Usuario → object`. Cada `super()` pasa al siguiente de esa lista.

- **Constructores:** cada uno llama a `super().__init__()` **antes** de imprimir, así que los mensajes salen de abajo arriba: Usuario, Moderador, Editor, Administrador.
- **Permisos:** igual, cada clase añade el suyo **al volver**: `leer` → `+ borrar comentarios` → `+ editar` → `+ gestionar usuarios`.

Pregunta 1: ¿por qué `"Configurando Usuario"` aparece una sola vez, si `Editor` y `Moderador` llaman los dos a `super().__init__()`? Porque `super()` no llama al padre directo, sino a la **siguiente clase del MRO**. El `super()` de `Editor` no va a `Usuario`, sino a `Moderador`; solo el `super()` de `Moderador` va a `Usuario`. `Usuario` aparece **una sola vez** en el MRO, así que su `__init__` se ejecuta una sola vez. Es la solución al **problema del diamante**.

Pregunta 2: cambia `Editor.permisos()` para que devuelva `["leer", "editar"]` sin `super()`. ¿Qué permiso pierde el administrador? Pierde `"borrar comentarios"` (resultado: `['leer', 'editar', 'gestionar usuarios']`). `Administrador.permisos()` llama a `Editor.permisos()`; como `Editor` ya no llama a `super()`, la cadena **se corta ahí** y nunca llega a `Moderador`, aunque esté en el MRO. Para que se sumen los permisos de todas las ramas, **todas** las clases de la cadena deben llamar a `super()`.

Pregunta 3: cambia a `class Administrador(Moderador, Editor)`. ¿Qué cambia? El MRO pasa a `Administrador → Moderador → Editor → Usuario → object`. Los permisos son los mismos, pero cambia el orden: `['leer', 'editar', 'borrar comentarios', 'gestionar usuarios']`. Los mensajes de los constructores salen como: Usuario, Editor, Moderador, Administrador. **El orden de los padres en la definición determina el MRO.**

> 🔁 **Refuerzo:** en el 7.4 el nombre de clase del `print` es **texto fijo** (`"Configurando Editor"`). Si usara `type(self).__name__`, al crear un `Administrador` las cuatro líneas dirían "Configurando Administrador", porque `type(self)` es la clase **real** del objeto, no la clase donde está escrito el código. En el 7.2 sí se quería la clase real, por eso allí se usa `type(self).__name__`.

Sección 8. Refactorización: de código espagueti a clases

Contexto: una agencia de viajes tiene un programa que **funciona**, pero está escrito todo seguido: listas con posiciones difíciles de recordar, muchos `if/elif` según el tipo de reserva y cálculos repetidos. Hay que **reorganizarlo con clases, herencia y polimorfismo**, sin cambiar lo que muestra por pantalla.

Parte A. Analizar el código original

```
# Cada reserva: [tipo, cliente, cantidad, precio_unitario, destino]
reservas = [
    ["vuelo", "Ana López", 2, 120.0, "Madrid-Roma"],
    ["hotel", "Luis Pérez", 3, 85.0, "Hotel Mar Azul"],
    ["tren", "Marta Gil", 1, 45.0, "Madrid-Sevilla"],
    ["vuelo", "Pablo Ruiz", 1, 300.0, "Madrid-Nueva York"],
    ["hotel", "Eva Sanz", 2, 140.0, "Hotel Centro"],
]

total = 0
for r in reservas:
    if r[0] == "vuelo":
        precio = r[2] * r[3]
        precio = precio + 25
        if r[3] > 200:
            precio = precio + r[2] * 30
        print("VUELO | " + r[1] + " | " + r[4] + " | " + str(r[2]) + " pasajeros |
" + str(round(precio, 2)) + " €")
    elif r[0] == "hotel":
        precio = r[2] * r[3]
        if r[2] >= 3:
            precio = precio * 0.9
        precio = precio + r[2] * 1.5
        print("HOTEL | " + r[1] + " | " + r[4] + " | " + str(r[2]) + " noches | "
+ str(round(precio, 2)) + " €")
    elif r[0] == "tren":
        precio = r[2] * r[3]
        print("TREN | " + r[1] + " | " + r[4] + " | " + str(r[2]) + " billetes | "
+ str(round(precio, 2)) + " €")
    total = total + precio

print("TOTAL AGENCIA: " + str(round(total, 2)) + " €")

# Contar reservas de más de 200 €: hay que repetir todos los cálculos
caras = 0
for r in reservas:
    if r[0] == "vuelo":
        precio = r[2] * r[3] + 25
        if r[3] > 200:
            precio = precio + r[2] * 30
    elif r[0] == "hotel":
        precio = r[2] * r[3]
        if r[2] >= 3:
```

```
            precio = precio * 0.9
        precio = precio + r[2] * 1.5
    elif r[0] == "tren":
        precio = r[2] * r[3]
    if precio > 200:
        caras = caras + 1

print("Reservas de más de 200 €: " + str(caras))
```

Reglas de precio que esconde el código

Tipo	Cantidad significa	Cálculo del precio
Vuelo	pasajeros	pasajeros × precio + 25 € de tasas. Si el billete cuesta más de 200 €, se suman 30 € por pasajero
Hotel	noches	noches × precio. Con 3 noches o más, 10 % de descuento. Después se suma 1.5 € por noche de tasa turística
Tren	billetes	billetes × precio

Preguntas previas

1. **¿Qué significan `r[2]` y `r[3]`? ¿Es fácil saberlo sin el comentario?** `r[2]` es la cantidad y `r[3]` el precio unitario. **No** es fácil saberlo sin el comentario, y además `r[2]` significa pasajeros, noches o billetes según el tipo. Son "índices mágicos".
2. **¿En cuántos sitios está escrita la regla del precio de un hotel?** En **2**: en el bucle principal y en el bucle que cuenta las reservas caras.
3. **Si mañana se venden cruceros, ¿cuántas partes habría que tocar?** Habría que añadir un `elif` en **cada bucle**: al menos 2 partes (y es fácil olvidarse de alguna).

Parte B, C y D. Las clases

⚠️ **Versión final corregida** (incluye el crucero de la Parte F)

```
class Reserva:
    def __init__(self, cliente, destino, cantidad, precio_unitario):
        self.cliente = cliente
        self.destino = destino
        self.cantidad = cantidad
        self.precio_unitario = precio_unitario

    def precio_base(self):
        return self.cantidad * self.precio_unitario

    def precio_total(self):
        return self.precio_base()

    def unidad(self):
        return "unidades"
```

```
    def __str__(self):
        tipo = type(self).__name__.replace("Reserva", "").upper()
        return (f"{tipo} | {self.cliente} | {self.destino} | "
                f"{self.cantidad} {self.unidad()} | {round(self.precio_total(),
2)} €")


class ReservaVuelo(Reserva):
    def unidad(self):
        return "pasajeros"

    def precio_total(self):
        precio = self.precio_base() + 25
        if self.precio_unitario > 200:
            precio += self.cantidad * 30
        return precio


class ReservaHotel(Reserva):
    def unidad(self):
        return "noches"

    def precio_total(self):
        precio = super().precio_total()
        if self.cantidad >= 3:
            precio *= 0.9
        return precio + self.cantidad * 1.5


class ReservaTren(Reserva):
    def unidad(self):
        return "billetes"


class ReservaCrucero(Reserva):
    def unidad(self):
        return "camarotes"

    def precio_total(self):
        return super().precio_total() * 1.12


class Agencia:
    def __init__(self):
        self.reservas = []

    def agregar(self, reserva):
        self.reservas.append(reserva)

    def mostrar(self):
        for r in self.reservas:
            print(r)
```

```
    def total(self):
        return sum(r.precio_total() for r in self.reservas)

    def contar_superiores_a(self, importe):
        return sum(1 for r in self.reservas if r.precio_total() > importe)
```

Explicación del diseño

- `Reserva` guarda lo común: los datos, `precio_base()`, `__str__` y un `precio_total()` por defecto.
- Cada subclase **solo** sobrescribe lo que le es propio. `ReservaTren` **no sobrescribe** `precio_total()` porque su precio es el base ("sobrescribir solo si lo necesita").
- `ReservaHotel` reutiliza `super().precio_total()` en lugar de repetir `cantidad * precio_unitario`.
- `type(self).__name__.replace("Reserva", "").upper()` convierte `ReservaVuelo` en `"VUELO"`.
- `Agencia` es **composición**: contiene una lista de reservas. No sabe de qué tipo es cada una; solo llama a `precio_total()` (polimorfismo).

Parte E y F. Programa principal y ampliación

Salida esperada

```
VUELO | Ana López | Madrid-Roma | 2 pasajeros | 265.0 €
HOTEL | Luis Pérez | Hotel Mar Azul | 3 noches | 234.0 €
TREN | Marta Gil | Madrid-Sevilla | 1 billetes | 45.0 €
VUELO | Pablo Ruiz | Madrid-Nueva York | 1 pasajeros | 355.0 €
HOTEL | Eva Sanz | Hotel Centro | 2 noches | 283.0 €
TOTAL AGENCIA: 1182.0 €
Reservas de más de 200 €: 4
```

Y tras añadir el crucero, cambian o se añaden estas líneas:

```
CRUCERO | Jorge Vidal | Crucero Mediterráneo | 1 camarotes | 1008.0 €
TOTAL AGENCIA: 2190.0 €
Reservas de más de 200 €: 5
```

```
agencia = Agencia()
agencia.agregar(ReservaVuelo("Ana López", "Madrid-Roma", 2, 120.0))
agencia.agregar(ReservaHotel("Luis Pérez", "Hotel Mar Azul", 3, 85.0))
agencia.agregar(ReservaTren("Marta Gil", "Madrid-Sevilla", 1, 45.0))
agencia.agregar(ReservaVuelo("Pablo Ruiz", "Madrid-Nueva York", 1, 300.0))
agencia.agregar(ReservaHotel("Eva Sanz", "Hotel Centro", 2, 140.0))

# Parte E: programa original
agencia.mostrar()
print(f"TOTAL AGENCIA: {round(agencia.total(), 2)} €")
print(f"Reservas de más de 200 €: {agencia.contar_superiores_a(200)}")
```

```
# Parte F: ampliación con crucero (sin tocar Reserva ni Agencia)
agencia.agregar(ReservaCrucero("Jorge Vidal", "Crucero Mediterráneo", 1, 900.0))
agencia.mostrar()
print(f"TOTAL AGENCIA: {round(agencia.total(), 2)} €")
print(f"Reservas de más de 200 €: {agencia.contar_superiores_a(200)}")
```

Qué fallaba en mi versión

- `ReservaCrucero` solo tenía `unidad()`: **faltaba** `precio_total()` con el 12 % de tasas portuarias (900 × 1.12 = 1008.0). Sin él, el crucero habría costado 900.0 y el total no sería 2190.0.
- Imprimía `TOTAL` y el contador **antes** de añadir el crucero, así que mostraban 1182.0 y 4. Para ver el efecto del crucero hay que mostrar `mostrar()`, `total()` y el contador **después** de agregarlo.

Restricciones del enunciado (se cumplen todas)

- No queda ningún `if/elif` que pregunte por el tipo de reserva.
- No se usa `isinstance()` para decidir cómo calcular el precio.
- El cálculo de cada tipo está escrito en **un único sitio** (su clase).
- Para añadir el crucero **no se modifica** ni `Reserva` ni `Agencia`.

Pregunta final: compara tu respuesta a la pregunta 3 de la Parte A con lo que has tenido que hacer en la Parte F. ¿Qué ha cambiado? En la Parte A se preveía tener que tocar **al menos dos partes** del programa (un `elif` en cada bucle). En la Parte F solo ha hecho falta **crear una clase nueva** (`ReservaCrucero`) con su `unidad()` y su `precio_total()`; `Reserva`, `Agencia` y el resto del programa **no se han modificado**. Es el beneficio del polimorfismo: el código que usa las reservas llama a `precio_total()` sin preguntar de qué tipo son, así que un tipo nuevo "encaja" sin cambios en el resto (principio *abierto a ampliación, cerrado a modificación*).

Sección 9. Ejercicio integrador: restaurante de comida a domicilio

Combina clases, herencia, polimorfismo, comprehensions, programación funcional y recursión.

Parte A. Jerarquía de clases

Enunciado: superclase `Plato` con `nombre` y `precio`, un método `precio_final()` que devuelve el precio y un `__str__` con el formato `Nombre - 12.50 €`, **usando** `precio_final()`. Subclases: `PlatoVegano` (10 % de descuento) y `PlatoGourmet` (recargo fijo de 3 €).

⚠️ Versión corregida

```
from functools import reduce


class Plato:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio
```

```
    def precio_final(self):
        return self.precio

    def __str__(self):
        return f"{self.nombre} - {self.precio_final():.2f} €"


class PlatoVegano(Plato):
    def precio_final(self):
        return self.precio * 0.90


class PlatoGourmet(Plato):
    def precio_final(self):
        return self.precio + 3
```

Qué fallaba (importante): mi `__str__` usaba `self.precio` en vez de `self.precio_final()`. Resultado: la carta mostraba el precio **sin** descuento ni recargo (`Ensalada quinoa - 11.00 €` en vez de `9.90 €`). El enunciado decía expresamente "usando `precio_final()`". Es el mismo patrón del 7.2: llama al **método**, no al atributo.

Parte B. Datos del día

```
pedidos = [
    {"cliente": "Ana",   "plato": PlatoVegano("Ensalada quinoa", 11.0),
"unidades": 2},
    {"cliente": "Luis",  "plato": Plato("Hamburguesa", 9.5),
"unidades": 1},
    {"cliente": "Marta", "plato": PlatoGourmet("Risotto trufa", 18.0),
"unidades": 1},
    {"cliente": "Ana",   "plato": Plato("Pizza margarita", 10.0),
"unidades": 3},
    {"cliente": "Pablo", "plato": PlatoVegano("Curry garbanzos", 12.0),
"unidades": 1},
]
```

Parte C. Comprehensions

```
# 1. Lista con el importe de cada pedido
importes = [round(p["plato"].precio_final() * p["unidades"], 2) for p in pedidos]

# 2. Set con los clientes distintos
clientes = {p["cliente"] for p in pedidos}

# 3. Diccionario nombre del plato -> precio final
precios_finales = {p["plato"].nombre: round(p["plato"].precio_final(), 2) for p in
pedidos}
```

- `pedidos` es una lista de **diccionarios**: se recorre con **una sola variable** (`p`) y se accede con `p["plato"]` y `p["unidades"]`.
- `precio_final` es un **método**: se llama sobre el objeto, `p["plato"].precio_final()`, no como función suelta.
- El set elimina el cliente repetido (Ana). Se ordena **al mostrarlo** con `sorted()`.

Parte D. Programación funcional

```
# 1. filter(): pedidos de platos veganos
veganos = filter(lambda p: isinstance(p["plato"], PlatoVegano), pedidos)
nombres_veganos = [p["plato"].nombre for p in veganos]

# 2. map() + reduce(): facturación total
facturacion = reduce(
    lambda a, b: a + b,
    map(lambda p: p["plato"].precio_final() * p["unidades"], pedidos)
)

# 3. Función que devuelve otra función
def crear_filtro_minimo(minimo):
    def filtro(pedido):
        return pedido["plato"].precio_final() * pedido["unidades"] >= minimo
    return filtro

filtro_20 = crear_filtro_minimo(20)
clientes_20 = [p["cliente"] for p in filter(filtro_20, pedidos)]
```

- `isinstance(objeto, Clase)` comprueba si el objeto es de esa clase (o de una subclase).
- `map` calcula el importe de cada pedido y `reduce` los suma.
- `crear_filtro_minimo` es una **clausura**: `filtro` recuerda `minimo`.

Parte E. Recursión

```
def contar_unidades(pedidos):
    if not pedidos:
        return 0
    return pedidos[0]["unidades"] + contar_unidades(pedidos[1:])
```

- **Caso base:** lista vacía → 0 unidades.
- **Caso recursivo:** unidades del primer pedido + las del resto.

Parte F. Informe final

```
print("=== CARTA DEL DÍA ===")
for p in pedidos:
    print(p["plato"])
```

```
print()
print("Importes por pedido:", importes)
print("Clientes distintos:", sorted(clientes))
print("Precios finales:", precios_finales)
print("Platos veganos pedidos:", nombres_veganos)
print(f"Facturación total: {round(facturacion, 2)} €")
print("Pedidos de 20 € o más:", clientes_20)
print("Unidades vendidas:", contar_unidades(pedidos))
```

Salida

```
=== CARTA DEL DÍA ===
Ensalada quinoa - 9.90 €
Hamburguesa - 9.50 €
Risotto trufa - 21.00 €
Pizza margarita - 10.00 €
Curry garbanzos - 10.80 €

Importes por pedido: [19.8, 9.5, 21.0, 30.0, 10.8]
Clientes distintos: ['Ana', 'Luis', 'Marta', 'Pablo']
Precios finales: {'Ensalada quinoa': 9.9, 'Hamburguesa': 9.5, 'Risotto trufa':
21.0, 'Pizza margarita': 10.0, 'Curry garbanzos': 10.8}
Platos veganos pedidos: ['Ensalada quinoa', 'Curry garbanzos']
Facturación total: 91.1 €
Pedidos de 20 € o más: ['Marta', 'Ana']
Unidades vendidas: 8
```

Pregunta final: añadir `PlatoInfantil` (precio fijo de 6 €)

Hay que cambiar solo una cosa: añadir la nueva clase.

```
class PlatoInfantil(Plato):
    def precio_final(self):
        return 6
```

Funciona sin tocar nada más:

- `__str__`, porque usa `self.precio_final()` y Python ejecuta la versión de la clase real.
- Las comprehensions de importes y de precios finales.
- `map()` + `reduce()` de la facturación y `crear_filtro_minimo`.
- `contar_unidades`, que solo mira las unidades.
- El informe y el `filter` de veganos: un `PlatoInfantil` no es `PlatoVegano`, así que no aparece, que es lo correcto.

Justificación: es **polimorfismo**. Todo el código trabaja con objetos `Plato` y llama a `precio_final()` sin preguntar de qué tipo son. La regla de cada precio vive en un solo sitio (su clase). Si el programa estuviera escrito con `if/elif` según el tipo, habría que tocar cada sitio donde se calcula el precio.

Sección 10. Programación modular

> **Nota:** el ejercicio 10.2 (mini aplicación con paquetes y subpaquetes) no está resuelto en estos apuntes. Aquí se incluye el **10.1** completo.

Ejercicio 10.1 · Del notebook a un proyecto modular

Contexto: *Hoteles Brisa* tiene tres hoteles (*Costa Azul*, *Sierra Nevada* y *Centro Histórico*). Un notebook genera el **informe de ingresos por hotel**:

1. Lee las reservas del día (CSV).
2. Descarta las reservas con noches o precio vacíos o mal escritos.
3. Descarta las reservas anuladas.
4. Calcula el importe: noches × precio por noche, con un **10 % de descuento para socios**, más una **tasa turística de 2,50 € por noche** (sin descuento).
5. Suma noches e ingresos por hotel, ordena de mayor a menor ingreso y calcula el **ingreso medio por noche**.
6. Muestra el informe y lo guarda en `informe_hoteles.csv`.

La dirección quiere que se ejecute **automáticamente cada noche**. El trabajo consiste en convertir el notebook en un **proyecto modular** que haga exactamente lo mismo.

Datos de entrada

```
id,cliente,hotel,tipo,noches,precio_noche,estado
R001, Carmen Vidal ,costa azul,socio,3,95.00,confirmada
R002,Andrés Molina,SIERRA NEVADA,general,2,120.00,anulada
R003,Lucía Ferrer,Costa Azul,general,5,95.00,confirmada
R004,Raúl Ortega,centro histórico,socio,,80.00,confirmada
R005,Nuria Campos,Sierra Nevada,general,4,120.00,check-out
R006,Tomás Reyes,centro histórico,general,1,8O.00,confirmada
R007,Irene Navarro,Centro Histórico,socio,2,80.00,confirmada
R008,Óscar Prieto,sierra nevada,socio,3,120.00,ANULADA
R009,Paula Serrano,Sierra Nevada,general,2,135.50,confirmada
```

> En `R006` el precio está escrito con la letra **O** en lugar de un cero (`8O.00`).

Parte A · Diagnóstico del notebook

Problema en el notebook	Dónde se ve	Consecuencia en producción
Si se ejecuta dos veces la celda 4, `r.append(...)` añade otra columna a cada reserva	Celda 4, `r.append(...)`	Datos duplicados; el resultado depende de cuántas veces se ejecute
Si se salta la celda 3, las anuladas entran en el cálculo	Celdas 3 y 4	Ingresos inflados sin ningún error visible

Problema en el notebook	Dónde se ve	Consecuencia en producción
Índices mágicos: `r[3]`, `r[4]`, `r[7]`	Celdas 4, 5 y 6	Nadie sabe qué significan; si cambia el orden de columnas todo se rompe
`0.90`, `2.50`, `"anulada"` y `"socio"` escritos a mano en varias celdas	Celdas 3, 4 y 7	Si cambia una regla hay que buscarla por todo el código
`es_socio()` definida en mitad del notebook y dependiente de la posición `r[3]`	Celda 4	No se puede reutilizar sin copiar y pegar
La validación solo comprueba que `int()` y `float()` no fallen	Celda 2	Líneas mal formadas (como `R011`) pasan sin error
Datos, limpieza, cálculo y salida mezclados, con variables globales	Todo el notebook	No se puede automatizar cada noche ni probar por partes

Parte B · Tabla de correspondencia

Celda del notebook	Módulo de producción	Función / clase / constante
1: cargar los datos	`extraccion.py` (texto en `main.py`)	`leer_lineas(texto)`
2: limpiar y descartar errores	`transformacion.py`	`parsear`, `es_valida`, `a_reserva`
3: quitar anuladas	`transformacion.py`	`filter` dentro de `transformar()`
4: calcular importes	`modelos.py`	`Reserva.importe_total` y `Reserva.es_socio`
5: agregar por hotel	`carga.py`	`InformeHoteles.por_hotel()` y `total()`
6: mostrar el informe	`main.py`	`mostrar_informe(resultado)`
7: guardar en CSV	`carga.py`	`exportar_csv(informe, ruta)`
`0.90`, `2.50`, `"anulada"`, `"socio"`, nombre del CSV	`config.py`	`DESCUENTO_SOCIO`, `TASA_TURISTICA`, `ESTADO_ANULADA`, `TIPO_SOCIO`, `FICHERO_SALIDA`

Parte C · El proyecto modular

Estructura

```
hoteles_etl/
├── main.py                ← punto de entrada
├── pruebas_manuales.py    ← pruebas de piezas sueltas
└── etl/                   ← paquete del pipeline
    ├── __init__.py        ← recepción del paquete
    ├── config.py          ← reglas de negocio y parámetros
    ├── modelos.py         ← clase Reserva
```

```
├── extraccion.py       ← E: obtener las líneas en bruto
├── transformacion.py   ← T: parsear, validar, convertir, filtrar
├── carga.py            ← L: clase InformeHoteles + exportar a CSV
└── orquestador.py      ← une E, T y L en el orden correcto
```

Reglas de importación

- **Dentro** del paquete `etl` se usan **imports relativos** (`from .config import TASA_TURISTICA`).
- Desde `main.py` se usan **imports absolutos** a través de la recepción (`from etl import ejecutar_pipeline, exportar_csv`).
- **Sin imports circulares.** Las dependencias van en un solo sentido: `config` → `modelos` → `transformacion` / `carga` → `orquestador` → `__init__` → `main`.

`etl/config.py`

```
DESCUENTO_SOCIO = 0.10
TASA_TURISTICA = 2.50
ESTADO_ANULADA = "anulada"
TIPO_SOCIO = "socio"
FICHERO_SALIDA = "informe_hoteles.csv"
```

`etl/modelos.py`

```
from .config import DESCUENTO_SOCIO, TASA_TURISTICA, TIPO_SOCIO


class Reserva:
    def __init__(self, id_reserva, cliente, hotel, tipo, noches, precio_noche,
estado):
        self.id_reserva = id_reserva
        self.cliente = cliente
        self.hotel = hotel
        self.tipo = tipo
        self.noches = noches
        self.precio_noche = precio_noche
        self.estado = estado

    @property
    def es_socio(self):
        return self.tipo == TIPO_SOCIO

    @property
    def importe_total(self):
        base = self.noches * self.precio_noche
        if self.es_socio:
            base = base * (1 - DESCUENTO_SOCIO)
        return round(base + TASA_TURISTICA * self.noches, 2)

    def __repr__(self):
```

```
        return (f"Reserva({self.id_reserva!r}, {self.cliente!r}, {self.hotel!r}, "
                f"{self.tipo!r}, {self.noches}, {self.precio_noche},
{self.estado!r})")
```

etl/extraccion.py

```
def leer_lineas(texto):
    return texto.split("\n")[1:]
```

etl/transformacion.py

```
from .config import ESTADO_ANULADA
from .modelos import Reserva


def parsear(linea):
    return [x.strip() for x in linea.split(",")]


def es_valida(campos):
    if len(campos) != 7:          # detecta líneas con campos de más o de menos
        return False
    try:
        int(campos[4])
        float(campos[5])
    except ValueError:
        return False
    return True


def a_reserva(campos):
    return Reserva(
        campos[0],
        campos[1].title(),
        campos[2].title(),
        campos[3].lower(),
        int(campos[4]),
        float(campos[5]),
        campos[6].lower(),
    )


def transformar(lineas):
    parseadas = list(map(parsear, lineas))                                  # map
    validas = [c for c in parseadas if es_valida(c)]                        #
comprehension
    errores = [c[0] for c in parseadas if not es_valida(c)]
    reservas = list(map(a_reserva, validas))
    anuladas = [r.id_reserva for r in reservas if r.estado == ESTADO_ANULADA]
```

```
    activas = list(filter(lambda r: r.estado != ESTADO_ANULADA, reservas))   #
filter + lambda
    return activas, errores, anuladas
```

etl/carga.py

```
from .config import FICHERO_SALIDA


class InformeHoteles:
    def __init__(self):
        self._reservas = []

    def cargar(self, reservas):
        self._reservas = list(reservas)

    def __len__(self):
        return len(self._reservas)

    def por_hotel(self):
        ingresos = {}
        noches = {}
        for r in self._reservas:
            ingresos[r.hotel] = round(ingresos.get(r.hotel, 0) + r.importe_total,
2)
            noches[r.hotel] = noches.get(r.hotel, 0) + r.noches
        ordenados = sorted(ingresos.items(), key=lambda x: x[1], reverse=True)
        return {h: (noches[h], t) for h, t in ordenados}

    def total(self):
        datos = self.por_hotel().values()
        return sum(n for n, _ in datos), round(sum(t for _, t in datos), 2)


def exportar_csv(informe, ruta=FICHERO_SALIDA):
    with open(ruta, "w", encoding="utf-8") as f:
        f.write("hotel,noches,ingresos\n")
        for hotel, (noches, ingresos) in informe.por_hotel().items():
            f.write(f"{hotel},{noches},{ingresos}\n")
    return ruta
```

etl/orquestador.py

```
from .extraccion import leer_lineas
from .transformacion import transformar
from .carga import InformeHoteles


def ejecutar_pipeline(texto):
```

```
    lineas = leer_lineas(texto)
    activas, errores, anuladas = transformar(lineas)
    informe = InformeHoteles()
    informe.cargar(activas)
    return {
        "leidas": len(lineas),
        "errores": errores,
        "anuladas": anuladas,
        "informe": informe,
    }
```

etl/__init__.py

```
from .orquestador import ejecutar_pipeline
from .carga import exportar_csv

__version__ = "1.0.0"
__all__ = ["ejecutar_pipeline", "exportar_csv"]
```

main.py

```
from etl import ejecutar_pipeline, exportar_csv

DATOS = """id,cliente,hotel,tipo,noches,precio_noche,estado
R001, Carmen Vidal ,costa azul,socio,3,95.00,confirmada
R002,Andrés Molina,SIERRA NEVADA,general,2,120.00,anulada
R003,Lucía Ferrer,Costa Azul,general,5,95.00,confirmada
R004,Raúl Ortega,centro histórico,socio,,80.00,confirmada
R005,Nuria Campos,Sierra Nevada,general,4,120.00,check-out
R006,Tomás Reyes,centro histórico,general,1,8O.00,confirmada
R007,Irene Navarro,Centro Histórico,socio,2,80.00,confirmada
R008,Óscar Prieto,sierra nevada,socio,3,120.00,ANULADA
R009,Paula Serrano,Sierra Nevada,general,2,135.50,confirmada"""


def mostrar_informe(resultado):
    informe = resultado["informe"]
    print("Reservas leídas:", resultado["leidas"])
    print("Reservas con errores:", resultado["errores"])
    print("Reservas anuladas:", resultado["anuladas"])
    print("Reservas válidas:", len(informe))
    print("Informe de ingresos por hotel:")
    for hotel, (noches, ingresos) in informe.por_hotel().items():
        print(f"  {hotel:<17}{noches:>3} noches {ingresos:>9.2f} €   "
              f"media/noche {ingresos / noches:>7.2f} €")
    noches_total, ingresos_total = informe.total()
    print(f"  {'TOTAL':<17}{noches_total:>3} noches {ingresos_total:>9.2f} €")
```

```
def main():
    resultado = ejecutar_pipeline(DATOS)
    mostrar_informe(resultado)
    ruta = exportar_csv(resultado["informe"])
    print(f"Informe guardado en {ruta}")


if __name__ == "__main__":
    main()
```

Parte D · Ejecución y comprobación

```
cd hoteles_etl
python main.py
```

Salida (idéntica a la del notebook)

```
Reservas leídas: 9
Reservas con errores: ['R004', 'R006']
Reservas anuladas: ['R002', 'R008']
Reservas válidas: 5
Informe de ingresos por hotel:
  Sierra Nevada      6 noches    766.00 €   media/noche  127.67 €
  Costa Azul         8 noches    751.50 €   media/noche   93.94 €
  Centro Histórico   2 noches    149.00 €   media/noche   74.50 €
  TOTAL             16 noches   1666.50 €
Informe guardado en informe_hoteles.csv
```

`informe_hoteles.csv`

```
hotel,noches,ingresos
Sierra Nevada,6,766.0
Costa Azul,8,751.5
Centro Histórico,2,149.0
```

Parte E · Probar piezas por separado

`pruebas_manuales.py`

```
from etl.transformacion import parsear, es_valida, a_reserva

print(es_valida(parsear("R010,Eva Lara,costa azul,general,2,95.00,confirmada")))
print(es_valida(parsear("R011,Eva Lara,costa azul,general,2,95,50,confirmada")))
print(es_valida(parsear("R012,Eva Lara,costa azul,general,,95.00,confirmada")))
```

```
print(a_reserva(parsear("R013, Eva Lara ,COSTA
AZUL,Socio,2,100.00,confirmada")).importe_total)

# ¿Por qué R011 habría pasado en el notebook?
# La línea tiene 8 campos porque "95,50" se parte en dos por la coma decimal.
# En el notebook solo se hacía int(c[4]) y float(c[5]): c[5] = "95" es un float
# válido, y c[6] = "50" se tomaba como estado (distinto de "anulada"), así que
# la reserva se aceptaba. Habría sumado 2 * 95 + 5 = 195.00 € en vez de
# 2 * 95.50 + 5 = 196.00 €: 1 € de menos, sin ningún aviso.
# Ahora es_valida() comprueba que haya exactamente 7 campos y la rechaza.
```

Salida

```
True
False
False
185.0
```

Conceptos que explican este ejercicio

- **ETL:** *Extract* (`extraccion.py`), *Transform* (`transformacion.py`), *Load* (`carga.py`). El `orquestador.py` los une en orden.
- **Funciones puras:** las de `transformacion.py` no usan variables globales ni modifican nada: reciben datos y devuelven resultados. Por eso se pueden probar solas (Parte E).
- **`@property`:** `importe_total` se **calcula al pedirlo**; no se guarda como columna. No se puede desincronizar si cambia el precio, y ejecutarlo dos veces no duplica datos (el problema de la celda 4).
- **`config.py`:** cada regla de negocio vive en **un solo sitio**. Para cambiar el descuento basta con editar una línea.
- **`__init__.py`:** es la "recepción" del paquete: expone solo `ejecutar_pipeline` y `exportar_csv`. `__all__` define qué se exporta con `from etl import *`.
- **`if __name__ == "__main__":`** hace que `main()` se ejecute solo al lanzar el fichero directamente, no al importarlo.
- **Imports relativos (`.config`)** funcionan dentro de un paquete; por eso hay que ejecutar **`main.py`** y no un fichero de dentro de `etl/`.

Resumen de errores que he cometido

Lista de fallos reales de mi código, ordenados por tema, para repasar antes del examen.

#	Ejercicio	Error	Cómo evitarlo
1	5.2	`ingresar` y `retirar` **calculaban pero no modificaban** `_saldo`	Para cambiar el estado: `self._saldo += x`. Un `return a + b` no guarda nada
2	5.2	Condición de `retirar` **invertida** y comisión sumada al saldo	Escribe la condición en castellano antes: "si importe + comisión ≤ saldo"

#	Ejercicio	Error	Cómo evitarlo
3	5.2	`total_cuentas` como lista cuando se pedía un contador	Lee el enunciado: "cuenta cuántas cuentas se han creado" → entero
4	4.5	La función "recursiva" llamaba a la **iterativa**	Una función recursiva debe llamarse **a sí misma**
5	7.2	`ficha()` usaba `cuota_base` en vez de `precio_mensual()`	Llama al **método** para que funcione el polimorfismo
6	9	`__str__` usaba `self.precio` en vez de `self.precio_final()`	Igual que el anterior: lee "usando X()" en el enunciado
7	8	`ReservaCrucero` sin `precio_total()` (faltaba el 12 %)	Releer la Parte F entera: pedía unidad **y** precio
8	8	Totales mostrados **antes** de añadir el crucero	Comprueba el orden de las instrucciones respecto a la salida esperada
9	7.1	Fórmula repetida en vez de `super().peaje()`; código tras un `return`	`super()` reutiliza; tras `return` no se ejecuta nada
10	7.4	`"Leer"` con mayúscula	Copia los textos **literalmente** del enunciado
11	7.3	Espacio sobrante en `"Equipo de oficina "`	Revisa los espacios en las cadenas que se concatenan
12	5.1, 6.3	`__str__` sin `€` ni `uds.`	Compara el formato con la salida esperada carácter a carácter
13	2.3	Mostrar un **set** sin ordenar	`sorted(conjunto)`
14	2.4	`"ULTIMAS PLAZAS"` sin tilde	Texto idéntico al enunciado
15	3.2	f-string con comillas dobles anidadas (solo vale en Python 3.12+)	Usa comillas **simples** dentro: `c['id']`
16	3.1, 3.3, 6.3	`True if cond else False`, `== False`, `and x["pasajero"]`	`cond`, `not cond`; sin condiciones de más
17	3.4, 4.4, 5.3	Textos de salida distintos a los pedidos	Copia el formato de salida esperado
18	7.3, 7.1, 7.2	`__init__` que solo llaman a `super().__init__()`	Si no añaden nada, **bórralos**

Patrón general: casi todos los fallos son de **detalle** (formato, un `return` mal puesto, un método que no se llama), no de concepto. En el examen, el último paso debe ser **comparar tu salida con la esperada línea a línea**.

Chuleta final

Clases y objetos

- `self.x = ...` en `__init__` → atributo de **instancia**. Variable en el cuerpo de la clase → atributo de **clase** (compartido).
- Para cambiar un atributo de clase: `Clase.x = ...`. `obj.x = ...` crea uno **nuevo** en el objeto.
- `_x` → convención de uso interno. `__x` → *name mangling* a `_Clase__x`.
- Un método que cambia el estado hace `self.x = ...`; no basta con `return`.

Comprehensions

Tipo	Sintaxis
Lista	`[expr for x in it if cond]`
Set	`{expr for x in it}`
Diccionario	`{k: v for x in it}`
Condicional en la expresión	`[a if cond else b for x in it]` (con `else`, al **principio**)
Filtro	`[x for x in it if cond]` (al **final**)
Dos `for`	`[y for fila in matriz for y in fila]` (mismo orden que bucles anidados)

Programación funcional

- `lambda args: expresion`: función anónima de una expresión.
- `map(f, it)` transforma; `filter(f, it)` selecciona; `reduce(f, it, ini)` acumula (hay que importarlo de `functools`).
- `map` y `filter` devuelven iteradores → envolver con `list(...)`.
- Una función puede **recibir** funciones y **devolverlas** (clausuras).

Recursión

- Caso base + caso recursivo que se acerca al base.
- Sin caso base → `RecursionError`.
- Cada llamada consume pila: los bucles escalan mejor.
- Estructuras anidadas → `isinstance(x, list)` + llamada recursiva.

Herencia

- `super().metodo()` → el **siguiente en el MRO**.
- `type(self).__name__` → clase **real** del objeto.
- `Clase.__mro__` → orden de búsqueda.
- Herencia múltiple cooperativa: **todas** las clases de la cadena deben llamar a `super()`.
- El orden de los padres en `class C(A, B)` determina el MRO.
- Herencia = "es un"; composición = "tiene un".
- Polimorfismo: mismo método, distinto comportamiento según la clase; el código que lo usa no pregunta por el tipo.

Programación modular

- Un **módulo** es un fichero `.py`; un **paquete** es una carpeta con `__init__.py`.
- Imports **relativos** dentro del paquete (`from .config import X`), **absolutos** desde fuera.
- Constantes de negocio en `config.py`; funciones puras para transformar; `main()` bajo `if __name__ == "__main__":`.
- Las dependencias van en una dirección: sin imports circulares.